



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

Département de génie informatique et génie logiciel

Cours INF1900 :
Projet initial de système embarqué

Travaux pratiques 7 et 8

Production de librairie statique et stratégie de débogage

Par l'équipe

No. 2834

Noms:

Renato Rezende
Thierry Poulin
Gabriel Bruyère
Eliott Bonnefoy

Date :
28 octobre 2024

Partie 1 : Description de la librairie

Décrire la librairie construite et formée (définitions, fonctions ou classes, utilité, etc.) pour que cette partie du travail soit bien documentée pour la suite du projet au bénéfice de tous les membres de l'équipe.

Classes :

- *Button*
- *Led*
- *Motor*
- *Pin*
- *PWMTimer*
- *Timer1*
- *Util*

Définitions utilitaires :

- *Communicator*
- *Debug*

Bouton

Une classe représentant un bouton mécanique branché à la carte-mère. Prend en charge la logique inverse au besoin. Utilisable avec les ISR ou non.

Attributs :

Privés :

```
const uint8_t DEBOUNCE_TIME_MS;
const uint8_t DEFAULT_BUTTON_MASK;
Pin    pin_;
uint8_t pinNumber_;
uint8_t buttonMask_;
bool    isInverseLogic_;
bool    pressedOnce_;
```

Constructeur :

```
Button(Pin&    pin,
        uint8_t buttonMask,
        bool    isInverseLogic);
```

Méthodes :

Publiques :

```
void enableExternalInterrupts();
void disableExternalInterrupts();
void catchRisingEdge();
void catchFallingEdge();
void catchAll();
bool isPressed();
bool isReleased();
```

Privées :

```
bool isMaskCorresponding();
bool isMaskNotCorresponding();
```

Notes :

Le flag d'interruption par défaut est INT0 sur PD2. Pour l'instant, il faut créer au préalable les Pin que l'on passe au constructeur (donc pas directement dans le constructeur). Ex. : Pin pin0 = Pin(...), puis Button(pin0, mask, 0).

Led

Une classe qui permet l'utilisation et le contrôle de DEL(s) à usage général.

Attribut privé :

Pin positivePin_, negativePin_ : La broche positive et celle négative pour la del.

Constructeur :

Le constructeur de la classe Led met les broches de la del en sortie.

Méthodes :

- Fonction toggle :

- Dépendance : Devraient être appelées après la fonction « turnRed » ou « turnGreen » mais pas immédiatement après la fonction « turnOff ».

Notes :

La fonction « toggle » de la classe Led permet d'inverser les valeurs des broches pour changer la couleur, rouge devient vert et inversement.

Motor

Une classe qui permet d'utiliser les moteurs du robot, en combinaison avec la minuterie 2 (Timer2) du microcontrôleur.

Attributs :

Publics :

```
static const uint8_t LOW_POWER
static const uint8_t MEDIUM_POWER
static const uint8_t MAX_POWER
```

Private :

```
Pin pinPWM_
Pin pinControl_
PWMTimer pwm_
```

Constructeurs / Destructeur :

Motor(Pin& pinPWM, Pin& pinControl, PWMTimer& pwm)

Motor(PortID portID, uint8_t pinPWM, uint8_t pinControl, PWMTimer& pwm)

Note : Les moteurs sont par défaut en mode marche avant.

~Motor() : Le destructeur coupe les moteurs.

Méthodes :

void **turnOff()** : arrêter le moteur.

Registres utilisés : OCR1A / OCR1B dépendant la pin PWM.

void **setPower**(uint8_t) : ajuster la puissance du moteur sur un échelle de 256.

Registres utilisés : OCR1A / OCR1B dépendant la pin PWM.

void **setBackwardMode**() : activer le mode marche arrière.

Broche utilisée : pinControl_.

void **setForwardMode**() : activer le mode marche avant.

Broche utilisée : pinControl_.

Notes :

Il faut soit créer les objets Pin avant de construire les moteurs, ou il faut passer les paramètres au constructeur alternatif, qui se charge de créer ses propres Pin composées. Également, pinPWM doit être PD6 ou PD7.

Pin

Une classe représentant une pin sur la carte-mère, associée à un port.

Attributs :

Private :

```
const PortID PORT_ID_;
const uint8_t MASK_;
const uint8_t PIN_NUMBER_;
const bool OLD_DDR_;
```

Constructeur :

```
Pin(PortID portID, uint8_t pinNumber, bool writeMode);
```

Méthodes :

Public :

```
void setReadMode();
    Set le DDR en mode entrée, soit à 0;
bool read() const;
void setWriteMode();
    Set le DDR en mode sortie, soit à 1;
void write(bool bit);
uint8_t getMask() const;
uint8_t getPinNumber() const;
PortID getPortID() const;
void toggle();
```

Private :

```
volatile uint8_t* getDDR();
uint8_t getPIN() const;
volatile uint8_t* getPORT();
void setDDR(bool bit);
```

Notes :

Contient un Enum Class PortID qui contient les valeurs A, B, C et D, qui correspondent donc au port sur lequel se trouve le pin. On utilise PortID dans la création d'un Pin. Aussi, getDDR, getPIN et getPORT utilisent le PortID pour retourner la valeur correspondante.

PWMTimer

Une classe qui permet la configuration et le contrôle du timer 2 en mode PWM, phase correct, 8-bit. Cette classe fournit des méthodes pour modifier la puissance et configurer les interruptions.

Constructeur :

Les constructeurs initialisent tous les registres de contrôle de la minuterie 2 à des valeurs qui resteront fixes lors de l'utilisation de la minuterie de génération de PWM -- à l'exception de OCR2A et OCR2B, qui sont à 0 par défaut. Ils peuvent optionnellement être initialisés à la création de l'objet PWMTimer.

Méthodes :

- Fonction setPowerA, setPowerB :
 - o Registre utilisé : Modifie OCR2A (pour setPowerA) ou OCR2B (pour setPowerB).
- Fonctions setPower :
 - o Registres utilisés : Lit le registre TCNT1 et modifie le registre OCR1A ainsi que le registre OCR1B pour « setDuration ».
 - o Dépendance : Cette fonction dépend des fonction setPower, setPowerA et setPowerB.
- Fonction startTimer :
 - o Registre utilisé : Modifie CS22, CS21 et CS20 de TCCR1B.
- Fonction stopTimer :
 - o Registre utilisé : Modifie CS22, CS21 et CS20 du registre TCCR1B.
 - o Dépendance : Devrait être appelées après la fonction « startTimer ».

Notes :

La classe PWMTimer permet de générer du PWM, phase correct, 8-bit, mais son utilisation nécessite les broches PD6 et PD7. Finalement, nous avons choisi d'utiliser le timer 2 de 8 bits pour la la génération de PWM, car 256 niveaux de puissance possibles sont amplement suffisants pour faire varier la vitesse.

Timer1

Une classe qui permet la configuration et le contrôle de la minuterie du timer 1 en mode CTC du robot. Cette classe fournit des méthodes pour sélectionner l'horloge, configurer les interruptions et définir des durées.

Attribut public :

Enum ClockSelect : un enum qui contient tous les modes de « clock source » utilisable par la minuterie. Cet enum permet d'éviter l'utilisation d'un « switch-case » pour assigner les bits CS12, CS11 et CS10 dans TCCR1B.

Attribut privé :

ClockSelect clock_ : la « clock source » actuelle de la minuterie.

uint16_t OCR1A_cycle_, OCR1B_cycle_ : la durée pour les comparaisons dans OCR1A/B.

Constructeur :

Le constructeur de la classe Timer1 initialise les registres de la minuterie à leur état par défaut à l'exception de TCCR1B qui est mis en mode CTC (WGM12 mis à 1).

Méthodes :

- Fonction setInterruptOCIEA, setInterruptOCIEB :
 - o Registre utilisé : Modifie OCIE1B ou OCIE1A du registre TIMSK1
 - o Dépendance : Devraient être appelées avant « startTimer ».
 - o Broches utilisées : PD4 (interruption OC1B), PD5 (interruption OC1A)
- Fonction setDuration, setDurationA :
 - o Registres utilisés : Lit le registre TCNT1 et modifie le registre OCR1A ainsi que le registre OCR1B pour « setDuration ».
 - o Dépendance : Cette fonction dépend de la fonction « convertMsToCycles » de la classe « Util ». Elle devrait également être appelées après « setClockSelect ».
- Fonction setDurationB :
 - o Registre utilisé : Modifie le registre OCR1B.
 - o Dépendance : Cette fonction dépend de la fonction « convertMsToCycles » de la classe « Util ». Elle devrait également être appelées après « setClockSelect ».
- Fonction startTimer :
 - o Registre utilisé : Modifie le registre TCNT1 et CS12, CS11 et CS10 de TCCR1B.
 - o Dépendance : Devrait être appelées uniquement après la fonction « setClockSelect » et les fonctions qui modifie la durée.
- Fonction stopTimer :
 - o Registre utilisé : Modifie WGM12, CS12, CS11 et CS10 du registre TCCR1B.
 - o Dépendance : Devrait être appelées après la fonction « startTimer ».

Notes :

La classe Timer1 permet une utilisation flexible du timer 1, mais son utilisation nécessite la déclaration d'au moins une ISR par l'utilisateur pour OCIEA et, de manière optionnelle, une ISR pour OCIEB. Les fonctions qui permettent de modifier la durée placée dans OCR1A, soit la borne supérieur du compteur de la minuterie ont une vérification qui est faite au cas que OCR1A soit changer pendant que le timer est actif. Dans un tel cas, si on assignerait une valeur plus petite que la valeur actuelle de TCNT1, la vérification permet d'éviter que le timer 1 se rende a sa valeur maximum avant d'être remis à zéro. Cela permet de conservé ainsi un temps raisonnable avant que l'ISR soit exécuté. Finalement, nous avons choisi d'utiliser le timer 1 de 16 bits pour la minuterie afin d'avoir une meilleure précision et des délais extrêmement courts, une résolution qui n'est pas nécessaire pour le pwm de toute façon.

Util

Une classe qui permet l'utilisation de constantes et de méthode générale pour utiliser dans les autres classes ou le main.

Notes :

Nous avons choisi de créer cette classe afin d'éviter la répétition de code pour des fonctions qui

seraient utilisées dans plusieurs classes ou pour des constantes qui seraient pratiques à plusieurs endroits différents au fur et à mesure de notre développement.

Communicator

Une collection de fonctions qui permet la communication via RS232, entre le robot et l'ordinateur.

Ce n'est pas une classe

Fonctions :

```
void setUsartRegisters();  
    UBRR0H, UBRR0L, UCSR0B (RXEN0, TXEN0, UCSZ02), UCSR0A (option),  
    UCSR0C (UCSZ01, UCSZ00, USBS0, UPM01, UPM00, UMSEL01, UMSEL00)  
void sendString(uint8_t length, char* stringPtr);  
void sendChar (uint8_t aChar);  
    Attend que le transmit buffer soit vide
```

Debug

Une collection de fonctions qui permet l'appel de fonctions d'affichage, afin d'aider le déverminage des programmes liés au robot.

Comme ce n'est pas une classe, elle ne contient pas d'attributs membres, ni de constructeur/destructeur.

Méthodes :

- Fonction debugPrint:
 - o Dépendance : Dépends des fonctions de Communicator.
 - o Broches utilisées : TXD0 & RXD0 (PD0, PD1)
- Fonction debugPrintStr:
 - o Dépendance : Dépends des fonctions de Communicator.
 - o Broches utilisées : TXD0 & RXD0 (PD0, PD1)

Notes :

La méthode debugPrint est surchargé afin de permettre l'écriture d'un caractère seul debugPrint(char), pour écrire un chiffre debugPrint(uint32_t). Pour écrire un string, comme la fonction prends deux arguments et qu'on utilise une macro, nous avons créé une autre fonction. Lorsque on utilise la commande make debug pour compiler l'exécutable, les fonctions DEBUG(x) et DEBUG_STR(x,y) sont transformées en debugPrint et debugPrintStr respectivement, permettant l'affichage de variables et strings à fins de déverminage.

Partie 2 : Décrire les modifications apportées au Makefile de départ

Le makefile de librairie est basé sur les Makefiles précédents, sauf que la commande all invoque l'archiveur au lieu du compilateur: AVR-AR, combiné avec les flags d'archiveur C, R et S. Ces modifications sont dans les variables ARC et ARCFLAGS respectivement. Afin de pouvoir accéder à tous les fichiers .h et .cpp dans les dossiers du répertoire lib, notre condition d'inclusion a été modifié en ajoutant un argument qui ajoute tous les sous-répertoires de lib qui contiennent potentiellement des fichiers .cpp et .h. Make Install a aussi été enlevé comme option, car le makefile ne doit servir qu'à créer une librairie.

Le makefile de code contient un argument de compilation qui permet de spécifier une librairie statique à utiliser lors de la compilation du code : -L ../lib pour trouver le répertoire

de la librairie, et `–lstatic` pour spécifier le nom de la librairie à utiliser.

Le makefile contient aussi un mode de compilation additionnel, `make debug`, qui prends les instructions de `make install`, mais en transformant les appels de `DEBUG(x)` (fonction d’affichage) en des appels de `debugPrint(x)`. Ces fonctions d’affichage utilisent la communication par RS232 pour afficher des caracteres sur la ligne de commande linux.